BRING YOUR OWN PROJECT (BYOP)

SMART ROUTE FINDER USING A STAR ALGORITHM

FUNDAMENTAL IN AIML

CSE(2001) (B11+B12+B13)

BY SHEKHAR RAJPUT

25BMR10013

FACULTY - DR. BANDLA PAVAN BABU

INTRODUCTION

Artificial Intelligence (AI) is now a key part of our everyday and emerging technology particularly in robots, automation and autonomous systems. One of the most important challenges in robot kinematics is path planning and obstacle avoidance where a robot is required to move from one position in workspace to another position without colliding with obstacle.

Efficient pathfinding is also critical for real world applications like warehouse bots, robotic vacuum cleaners, delivery robots, and driverless cars. It must be capable of processing its environment, detecting obstacles, and finding the shortest route to a target point.

This work deals with the problem of robot navigation in a room with static obstacles using the A* (A-star) search algorithm*, which is one of the most popular path-finding algorithms in Artificial Intelligence for shortest path problems.

The A* algorithm uses both the true cost to reach the current point and the heuristic cost to reach the goal from the current point, this allow it to get the "best of both worlds".

The aim of this project is to develop and simulate an exhaustive search based intelligent robot path planning system for the robot to navigate from a start location to an end location while not colliding with obstacles and with the minimal travelling cost.

PROBLEM STATEMENT

The problem studied in this paper is that of an autonomous robot navigating in an indoor environment with static obstacles (walls, furniture, etc.) and blocked cells.

The robot needs to find the shortest collision free path from a given start position to a target position.

This is a problem of great interest in robotics, warehouse automation, delivery systems, and autonomous navigation.

FUNCTIONAL REQUIREMENTS

The system must be able to:

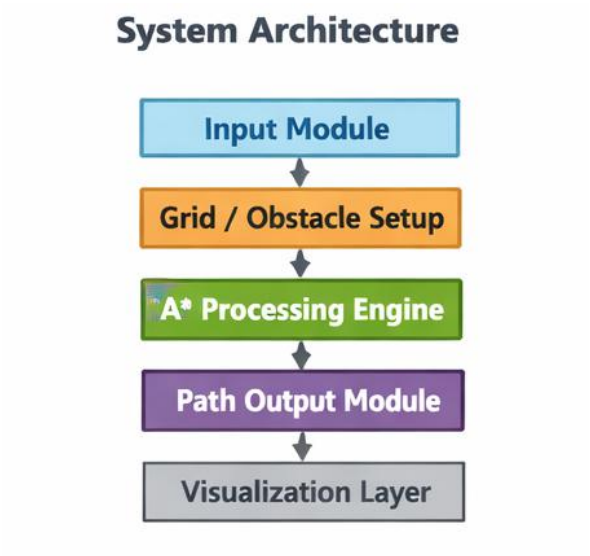
- accept a **start node**
- accept a **goal node**
- allow **obstacle placement**
- calculate shortest path using *A algorithm**
- display the calculated path
- allow reset and rerun

NON-FUNCTIONAL REQUIREMENTS

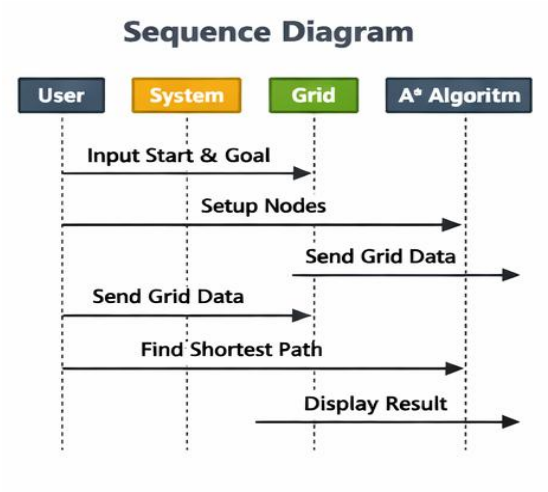
The system should satisfy the following:

- **performance:** fast path calculation
- **usability:** simple and easy interface
- **scalability:** larger grid support
- **maintainability:** modular code
- **reliability:** accurate path detection

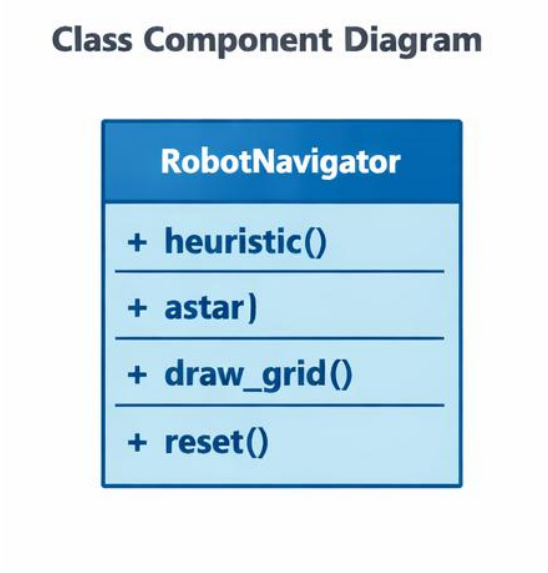
SYSTEM ARCHITECTURE



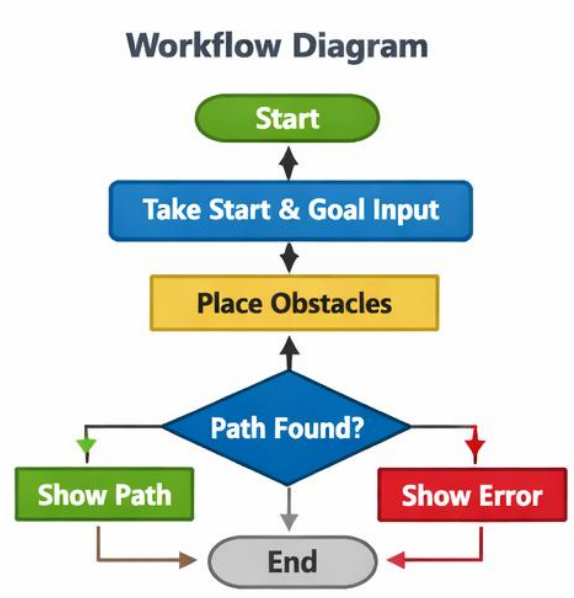
SEQUENCE DIAGRAM



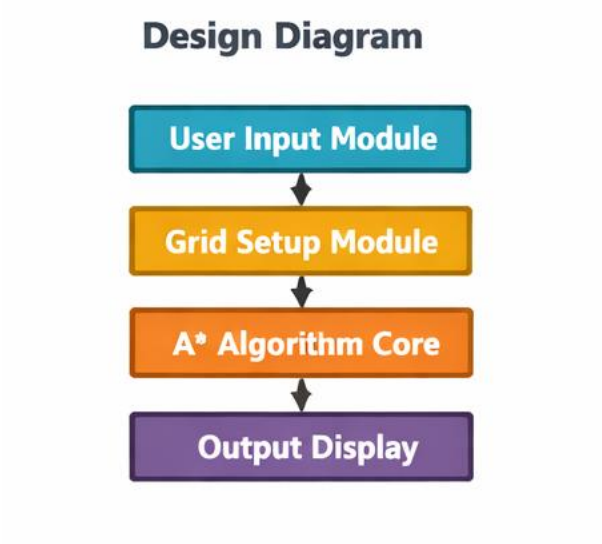
CLASS COMPONENT DIAGRAM



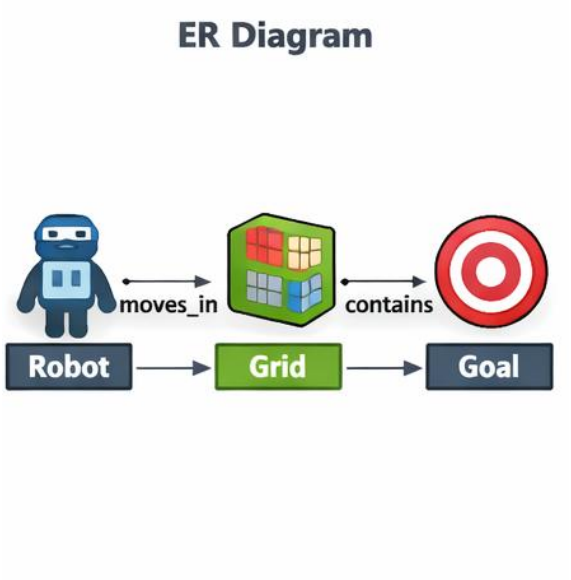
WORKFLOW DIAGRAM



DESIGN DIAGRAM



ER DIAGRAM



DESIGNS, DECISIONS AND RATIONALE

The A* algorithm* was chosen as it is one of the fastest shortest-path algorithms known in the field of Artificial Intelligence.

The Manhattan Distance heuristic was selected as we can only move in grid-based up, down, left and right directions.

Python was chosen for ease of use and availability of AI libraries.

INPUT

astarfinal.py 1

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > ...

```python
import pygame
from queue import PriorityQueue

# SETTINGS
WIDTH = 600
ROWS = 30

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GREY = (200, 200, 200)
GREEN = (0, 255, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
ORANGE = (255, 165, 0)
PURPLE = (128, 0, 128)

pygame.init()
WIN = pygame.display.set_mode((WIDTH, WIDTH))
pygame.display.set_caption("A* Pathfinding")

# NODE CLASS
class Node:
    def __init__(self, row, col, width):
        self.row = row
        self.col = col
        self.x = col * width   # ✅ FIXED
        self.y = row * width   # ✅ FIXED
        self.color = WHITE
        self.width = width

    def draw(self, win):
        pygame.draw.rect(win, self.color, (self.x, self.y, self.width, self.width))


# GRID
def make_grid():
    grid = []
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python Python 3.14 (64

astarfinal.py 1

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > get_neighbors

```python

# GRID
def make_grid():
    grid = []
    gap = WIDTH // ROWS
    for i in range(ROWS):
        grid.append([])
        for j in range(ROWS):
            grid[i].append(Node(i, j, gap))
    return grid


def draw(win, grid):
    win.fill(WHITE)

    for row in grid:
        for node in row:
            node.draw(win)

    gap = WIDTH // ROWS
    for i in range(ROWS):
        pygame.draw.line(win, GREY, (0, i * gap), (WIDTH, i * gap))
        pygame.draw.line(win, GREY, (i * gap, 0), (i * gap, WIDTH))

    pygame.display.update()


# HEURISTIC
def h(p1, p2):
    x1, y1 = p1
    x2, y2 = p2
    return abs(x1 - x2) + abs(y1 - y2)

```

```python
# NEIGHBORS
def get_neighbors(node, grid):
    neighbors = []
    r, c = node.row, node.col

    if r < ROWS - 1 and grid[r + 1][c].color != BLACK:
        neighbors.append(grid[r + 1][c])
    if r > 0 and grid[r - 1][c].color != BLACK:
        neighbors.append(grid[r - 1][c])
    if c < ROWS - 1 and grid[r][c + 1].color != BLACK:
        neighbors.append(grid[r][c + 1])
    if c > 0 and grid[r][c - 1].color != BLACK:
        neighbors.append(grid[r][c - 1])

    return neighbors
```

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

```python

# A* ALGORITHM
def astar(draw_func, grid, start, end):
    count = 0
    open_set = PriorityQueue()
    open_set.put((0, count, start))

    came_from = {}
    g_score = {node: float("inf") for row in grid for node in row}
    g_score[start] = 0

    f_score = {node: float("inf") for row in grid for node in row}
    f_score[start] = h((start.row, start.col), (end.row, end.col))

    while not open_set.empty():
        current = open_set.get()[2]

        if current == end:
            while current in came_from:
                current = came_from[current]
                current.color = BLUE
                draw_func()
            return True

        for neighbor in get_neighbors(current, grid):
            temp_g = g_score[current] + 1

            if temp_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = temp_g
                f_score[neighbor] = temp_g + h(
                    (neighbor.row, neighbor.col),
                    (end.row, end.col)
                )
```

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

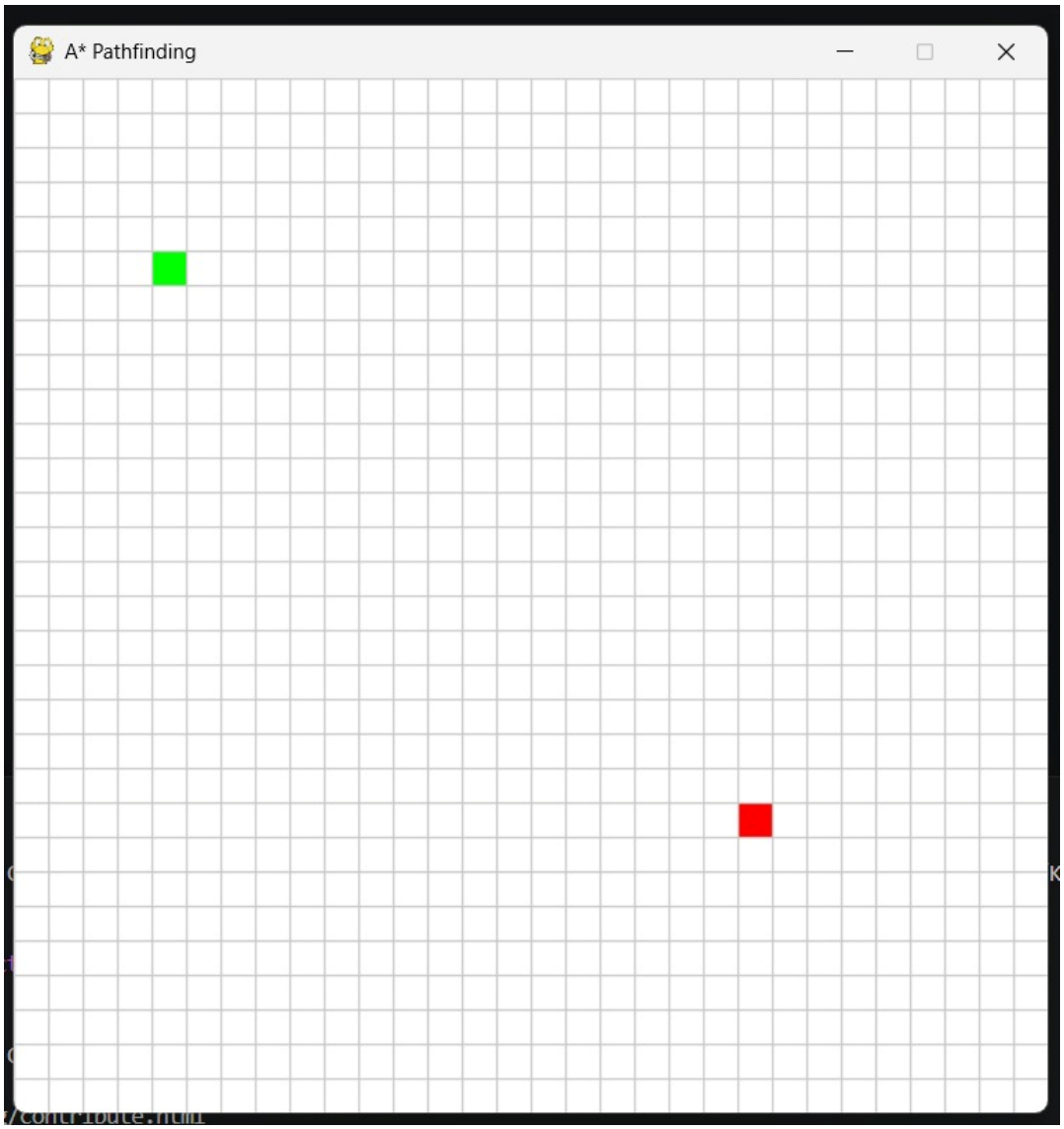
```python
def astar(draw_func, grid, start, end):
            if temp_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = temp_g
                f_score[neighbor] = temp_g + h(
                    (neighbor.row, neighbor.col),
                    (end.row, end.col)
                )

                count += 1
                open_set.put((f_score[neighbor], count, neighbor))
                neighbor.color = ORANGE

        if current != start:
            current.color = PURPLE

        draw_func()

    return False


# MAIN
def main():
    grid = make_grid()
    start = None
    end = None

    run = True
    while run:
        draw(WIN, grid)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
```

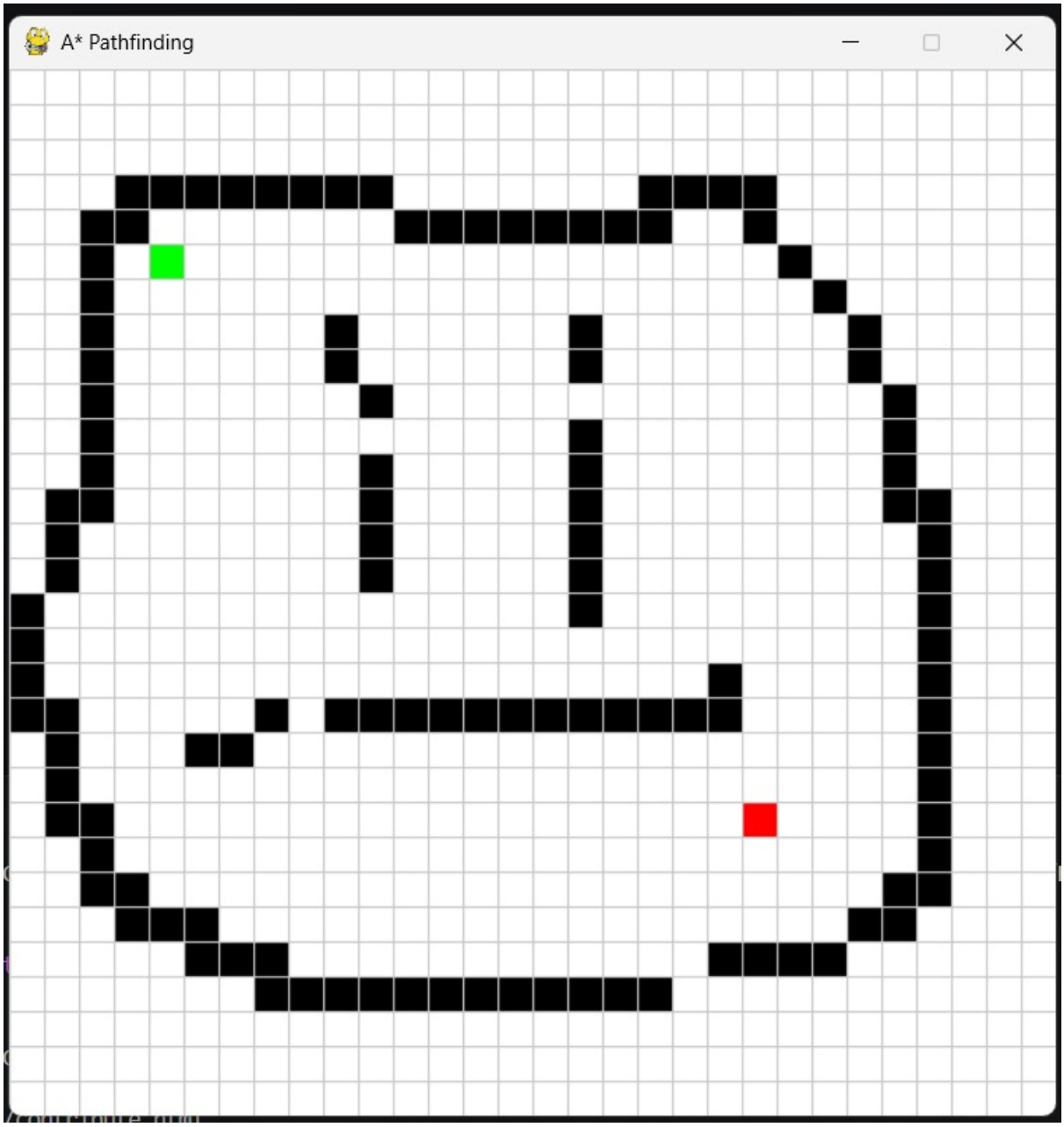
```python
# MAIN
def main():
    grid = make_grid()
    start = None
    end = None

    run = True
    while run:
        draw(WIN, grid)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE and start and end:
                    astar(lambda: draw(WIN, grid), grid, start, end)

                if event.key == pygame.K_c:
                    start = None
                    end = None
                    grid = make_grid()

            # LEFT CLICK
            if pygame.mouse.get_pressed()[0]:
                pos = pygame.mouse.get_pos()
                gap = WIDTH // ROWS
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]
```

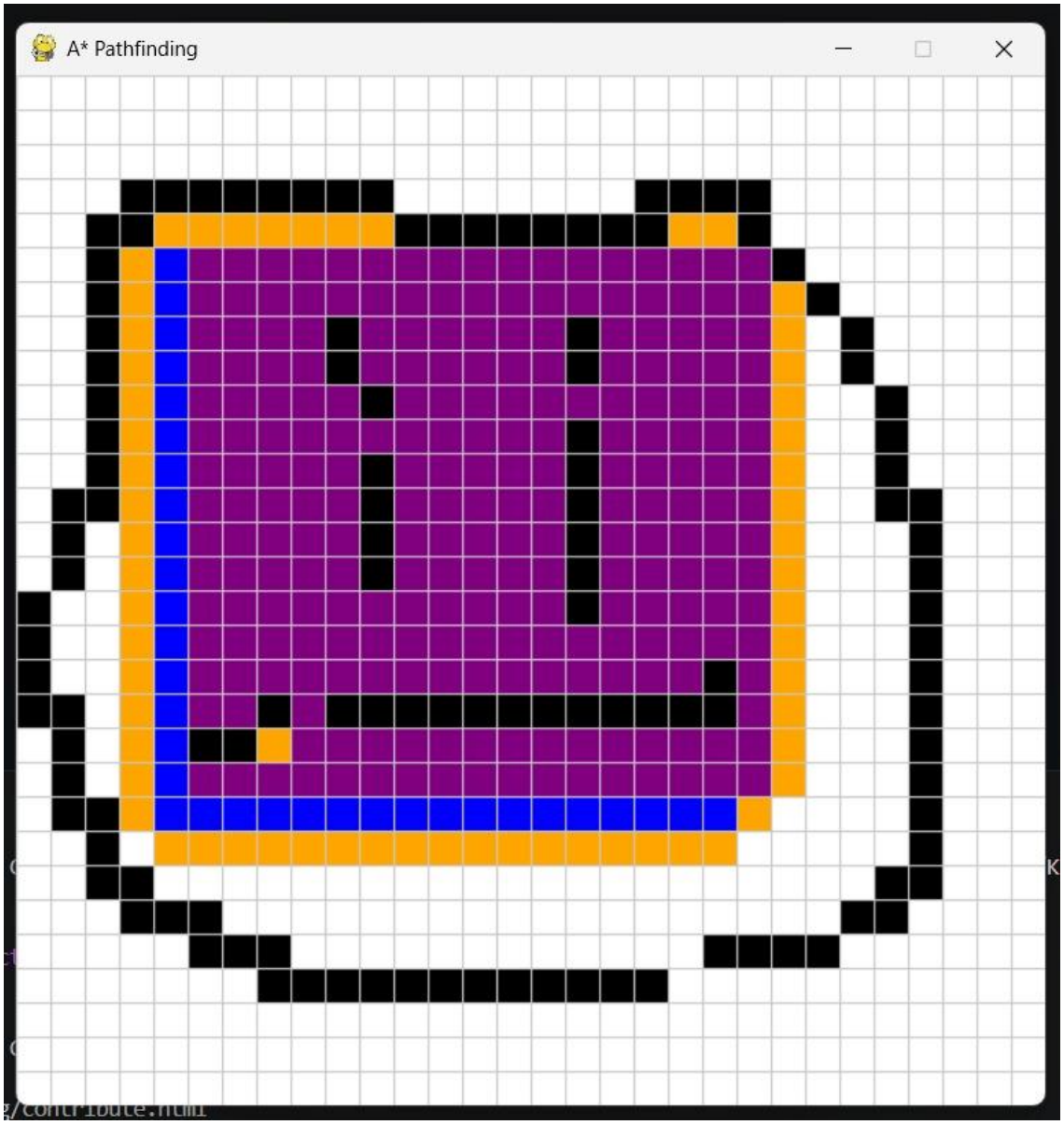
C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

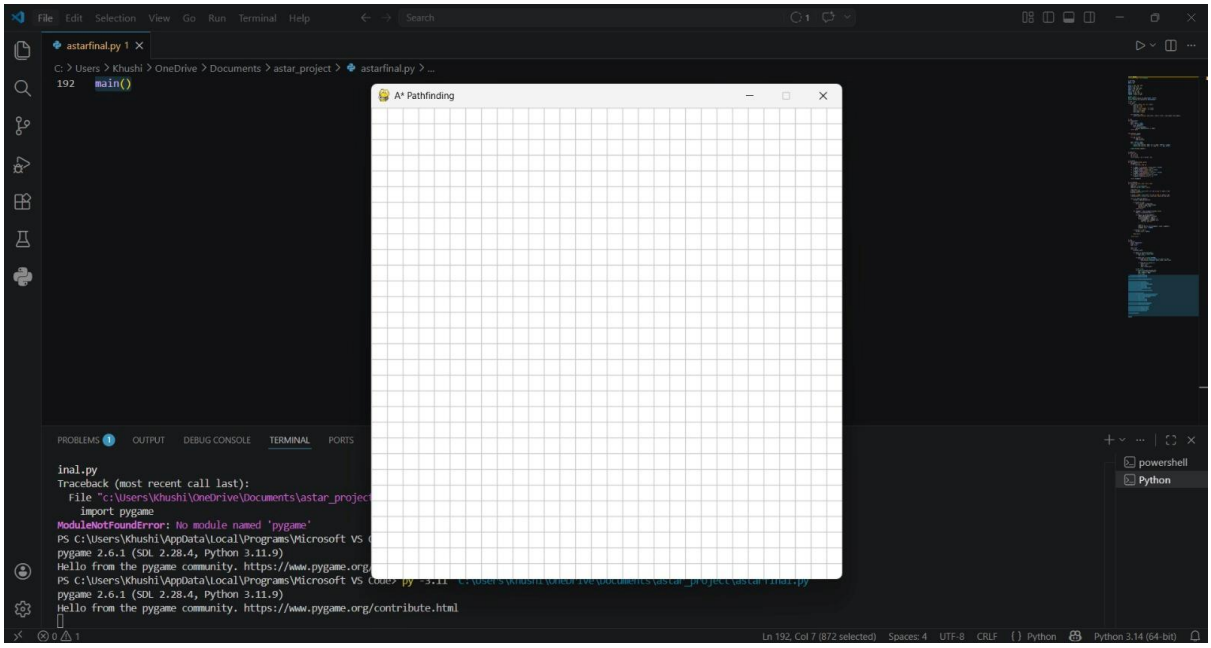
```python
def main():
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]

                if not start:
                    start = node
                    node.color = GREEN
                elif not end:
                    end = node
                    node.color = RED
                else:
                    node.color = BLACK

            # RIGHT CLICK
            elif pygame.mouse.get_pressed()[2]:
                pos = pygame.mouse.get_pos()
                gap = WIDTH // ROWS
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]
                node.color = WHITE

                if node == start:
                    start = None
                elif node == end:
                    end = None

    pygame.quit()


main()
```

OUTPUT




A* Pathfinding


A* Pathfinding



IMPLEMENTATION DETAILS

The project is implemented using:

- Python
- Pygame
- Heap Queue (heapq)

Core formula used:

$$f(n) = g(n) + h(n)$$

Where:

- g(n) = distance already travelled
- h(n) = estimated distance to target

TESTING APPROACH

The system was tested for:

- no obstacle case
- multiple obstacle case
- blocked path case
- edge positions

- no valid path

CHALLENGES FACED

- obstacle handling
- shortest path accuracy
- repeated node checking
- grid visualization

LEARNING AND KEY TAKEAWAYS

This project helped in understanding:

- AI search algorithms
- heuristic functions
- robotics path planning
- real-world problem solving

FUTURE ENHANCEMENTS

Possible future upgrades:

- moving obstacles
- animated robot
- diagonal movement
- voice-controlled navigation
- 3D room simulation

REFERENCES

- A* Algorithm Concepts
- Python Documentation
- Pygame Documentation
- Class Lecture Notes

CONCLUSION

With this project we want to give the proof of concept A (A-Star) algorithm can be used to solve shortest track/path problem in a two-dimensional grid. By mixing algorithmic logic with animated visualization, the system makes it easy to understand how intelligent pathfinding is used in real applications.

The project emphasis the relevance of heuristic based search method for increasing efficiency and accuracy. The implementation illustrates how A" makes an intelligent search of nodes and calculate the best path circumventing obstacles.

In general, this project has been very useful to refresh and reinforce some important concepts related to artificial intelligence, problem solving and python programming. It also demonstrated the importance of visualization to make complex algorithms more friendly to the user.